BUBBLE-BLUE

Private group messaging
over an open Bluetooth LE mesh

Paul-Maxence Baraton, Nadjib Achir, Philippe Jacquet (Inria)
Wilfred Doré (Qualcomm)

NetMob 2026 · Niterói arXiv:2509.00967

30 minutes

1. Why an open mesh with a private group
2. How a frame travels, and who is allowed to read it
3. How two members share a key, and how we drop one
4. What six boards measured: a room, a building, a working day
5. What broke, what we do not claim, what comes next

The stand is next to the poster: send, exclude, restart.

The settings that need this

Local communication still collapses onto a Wi-Fi access point, or onto a phone OS that will not host a third-party mesh.

- a crowd at an event: the infrastructure is saturated
- a site without coverage
- an emergency response: the infrastructure is absent or distrusted

Bluetooth is on every device and needs no operator.

The question is not “can two phones see each other”.
It is: **can a stranger carry a conversation they will never read?**

Two things existing meshes merge

The mesh (open)

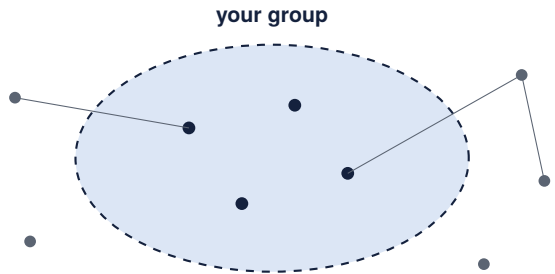
any board may join and relay, including traffic it cannot decrypt.

no pairing, no bond

The bubble (private)

a group exists only for the holders of a key.

membership is cryptographic, not a Bluetooth relationship



Where BUBBLE-BLUE sits

System	Data plane	Backbone	Open relay
SIG Mesh	**managed flood**	**provisioned**	**no**
BitChat	**GATT flood**	**none**	**n/a**
Briar	**RFCOMM, Wi-Fi, Tor**	**contacts**	**no**
Reticulum	**multi-transport**	**identities**	**no**
BUBBLE-BLUE	**LE CoC**	**elected CDS**	**yes**

- SIG Mesh: a relay must hold the network key
- BitChat, Bridgefy: a crowd-scale BLE flood, no elected backbone
- Briar: only pre-paired contacts

None of the others lets a node sit in an elected backbone and relay for a group it does not belong to.

What this talk adds

The original note gave the algorithms and unit-disk simulations. This work:

1. a BLE transport (L2CAP connection-oriented channels, no pairing)
2. a 26-byte frame and a revocation protocol that reaches several hops
3. measurements on six Qualcomm boards, in a room and across a building

Not a new routing theory.
A protocol that runs on hardware you can pick up.

Links without pairing

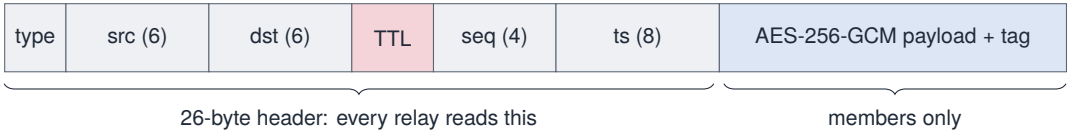
A node advertises a compact presence record:

- an anonymous identifier
- a relay flag
- the address at which it accepts connections

Neighbours open an L2CAP CoC at security level *low*: **no pairing, no bond, no allow-list**.

- Company ID `0xFFFF`: the SIG test value (this is a prototype)
- HELLO every 5 s; a neighbour silent for 15 s is dropped
- membership is a 32-byte group key, not a Bluetooth relationship

One frame, two planes



- relays decrement TTL and forward; they never see the text
- header authenticated as associated data, **except TTL**
- up to 470 bytes in one frame

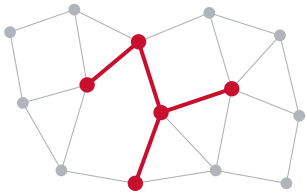
Who relays: an elected backbone

BLE has no native broadcast on a connection.
Every local flood is a sequence of unicasts, so flooding everyone is expensive.

Each node applies the Wu–Li rule to its two-hop view:

- marked nodes form a connected dominating set
- only they rebroadcast
- duplicates dropped on (source, sequence)

The backbone adds no path stretch.
Default TTL 10 spans about twenty nodes (simulation).



backbone, elected locally

Group traffic

With K_e the group key of epoch e, every frame gets its own key:

$$K_{\text{pkt}} = \text{HKDF}(K_e;\ src \,\|\, seq \,\|\, ts)$$

$$c = \text{GCM}_{K_{\text{pkt}}}(m;\ header)$$

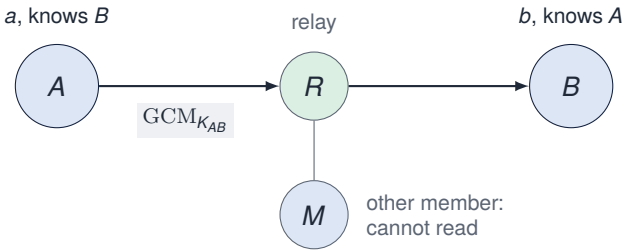
- a zero GCM nonce is never reused under one key
- a relay without K_e can forward; it cannot read or forge
- AES-256-GCM on these boards: 30k–173k pkt/s — far above the link

The bottleneck is the Bluetooth stack, not the cryptography.

Peer to peer inside the group

Each board A holds an X25519 secret a.
Every 30 s it floods a signed ANNOUNCE binding its address to $A = aG$. Two members then share a key without ever having met:

$$K_{AB}^{(e)} = \mathrm{HKDF}(aB,\ e) = \mathrm{HKDF}(bA,\ e) \qquad \text{since } aB = bA = abG$$



A direct message crosses the mesh like any frame. Relays and other members forward it; they cannot read it.

Removing a member V

The coordinator C (smallest anonymous identifier) draws K_{e+1} and a one-time key k, then floods one frame. Each remaining member i gets k wrapped under the pairwise key it shares with C:

$$w_i = \mathrm{HKDF}(K_{Ci}^{(e)};\ src \,\|\, seq \,\|\, ts)$$
$$case_i = tag_i \,\|\, \mathrm{GCM}_{w_i}(k) \qquad (i \neq V)$$
$$body = \mathrm{GCM}_k(e{+}1 \,\|\, K_{e+1} \,\|\, V \,\|\, \sigma_C)$$

σ_C: Ed25519 signature over the epoch, the key and the revoked key. There is no case for V.

The rekey frame on the air



- relays, V included, forward the frame unchanged
- V keeps relaying the new traffic; it cannot read it
- an observer can count the remaining members (one case each)
- wait one ANNOUNCE period (30 s) after a join before revoking

What each party learns

	radio listener	relay, not a member	member
addresses, hops left, time	yes	yes	yes
size and timing of traffic	yes	yes	yes
text of a group message	no	no	yes
forge or alter a message	no	no	yes

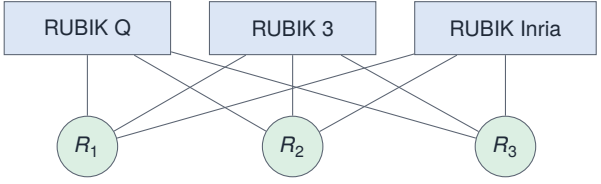
Accepted limits of this prototype

- payload privacy, not location privacy (the Bluetooth address is stable)
- a non-member relay can drop, delay, or rewrite TTL
- an active outsider can flood junk or pre-empt a (source, sequence)
- a rogue member can grind its identifier to become coordinator

Six boards, two roles

	Thundercomm RUBIK Pi 3	Arduino UNO Q
Role on the stand	**screen**	**relay**
SoC	**Qualcomm QCS6490**	**Qualcomm QRB2210**
OS, kernel	**Ubuntu 24.04, 6.8**	**Debian 13, 7.0**
BlueZ	**5.72**	**5.82**
Controller	**BCM4345C5 (BT 5.0)**	**WCN399x (BT 5.1)**

On this UNO Q image the controller cannot dial while it is peripheral, so relays always dial and screens never do.



On a table the graph is $K_{3,3}$: every message between screens crosses a relay that is not in the group.

On the stand

75/75
receptions on the five other boards

4.4 s
rebooted screen reads the group after daemon start

130 s
cut controller reads again, no reboot of the board

427/429
six-node manual soak (first and last lost)

no
revoked board reads the next message (2/2)

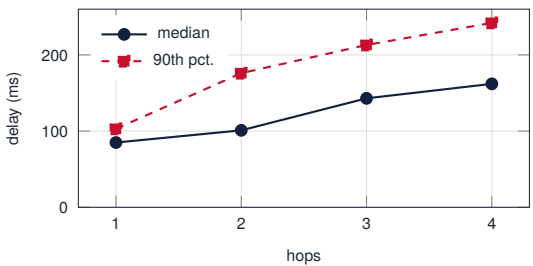
10 / 30
sustained msg/s UNO Q / RUBIK

AES-GCM is 30k–173k pkt/s. Above 10 msg/s the UNO Q runs out of L2CAP signalling identifiers — a kernel leak, not the crypto.

Four hops across a building

Boards about 15 m apart, one relay off, on battery.
The mesh formed a line by itself. On-board logs; clocks within 50 ms.



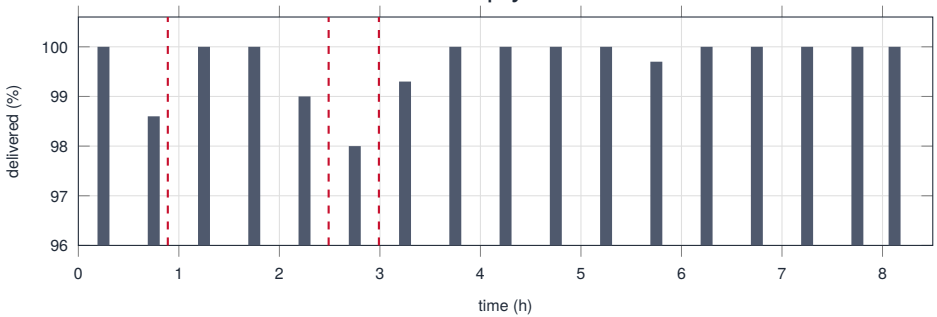


- far end: **76/79**
- the three misses: line still forming
- last link held to ∼30 m
- sender's only relay unplugged: **95 s** dark

A line has no detour. Mobility looked like ordinary link churn.

8.2 hours on one charge

Stand on battery, one message every 30 s from RUBIK Q.
988 messages, 4 915 expected receptions. We stopped the run; the batteries were not empty.



Dashed: controller reloads on two UNO Q relays (about two minutes deaf, no reboot). Screens 100%, relays 99.3%. Median 80 ms, 99th percentile 188 ms. 6/6 still up.

Three defects, found by measuring

1. **Shared anti-replay window.** ANNOUNCE and data used one window: a data frame with an ANNOUNCE's sequence was dropped everywhere. *Fixed, regression test.*
2. **Split-brain coordinator.** On a sparse 30-node graph, four nodes elected themselves. *Fixed: one membership view, regression test.*
3. **Sequence numbers die on restart.** After the sender's daemon restarted, every receiver's window silently dropped it. Only a receiver restarted afterwards accepted the new messages. *Open: sequence numbers must survive a restart.*

Defect 3 appeared at the end of the 8.2 h run, when the sender daemon was restarted on the measuring laptop.

TTL is a hop budget, not a node count

Shipped forwarder, random geometric graphs, three sources per draw.

nodes	degree	eccentricity	min TTL	delivery @ TTL 10
10	3.9	4	4	100%
20	4.3	8	8	100%
30	3.3	11	11	93%
50	5.2	13	13	94%
100	5.2	19	19	23%

The minimum TTL that reaches everyone equals the worst source's eccentricity, on every draw.

Revocation, $n = 6$ to 30: no stranded member, no leak to the excluded one.

From a stand to a contact graph

Every board already logs every link change with a timestamp. That log **is** a contact graph.

- the unit-disk simulations can be replayed on real traces
- then we can see how CDS election and revocation behave when *people* move, not boards on a table
- the four-hop walk already looked like ordinary link churn

Also open: an Android port of the CoC transport, and the energy of AES-GCM versus the radio.

What to try on the stand

Three RUBIK Pi 3 screens, three UNO Q relays.

1. **Send** from one screen: the message crosses a relay that is not in the group.
2. **Exclude** a screen or a relay: it keeps relaying; the next message no longer appears on it.
3. **Restart** the mesh: watch the links and the group come back.

Takeaways

1. An open BLE mesh with private groups is implementable without pairing.
2. Relays read a 26-byte header. Members share K_e for the group, and K_{AB} for traffic the rest of the group must not see.
3. Revocation is one signed flood, wrapped under those pairwise keys.
4. On six boards: 75/75 in the room, 76/79 at four hops, 8.2 h on battery at 99.7% of deliveries.
5. The limit we hit is the BlueZ stack of the UNO Q, and a sequence counter that must survive a restart.

Thank you

Poster and stand are next door.
arXiv:2509.00967

Boards provided by the Qualcomm University Platforms program.
paul-maxence.baraton@inria.fr

Backup: joining, four clocks

A new node is not “in the group” the moment it appears.

1. presence: one advertising interval
2. channel: one L2CAP handshake
3. HELLO: it enters neighbours’ tables
4. ANNOUNCE ($\leq$30 s): its public keys reach beyond one hop

Only after (4) can it be revoked safely.
The screens therefore show **link state** and **group state** separately.

Backup: why the graph is $K_{3,3}$

- two screens never dial each other
- two UNO Qs cannot: the controller cannot dial while it is peripheral on any link
- so every path alternates screen and relay

The same UNO Q kernel leaks one L2CAP signalling identifier per received credit (upstream fix: `6e1930ece855`).
Links are recycled before the pool of 254 runs out.
A watchdog reloads a controller that stops answering, without rebooting the board.

Backup: Bridgefy and the crowd

Albrecht et al., CT-RSA 2021, on Bridgefy in large-scale protests:

- impersonation
- tracking
- denial of service

A mesh used in a crowd must withstand those.
BUBBLE-BLUE answers impersonation of *payloads* (GCM + signatures).
It does **not** answer tracking: the Bluetooth address is stable on purpose, so a neighbour can be redialled without a bond.